

Chapter 1. 신경망 복습 (~74p)

1.1. 수학과 파이썬 복습

1.1.1. 벡터와 행렬

벡터란?

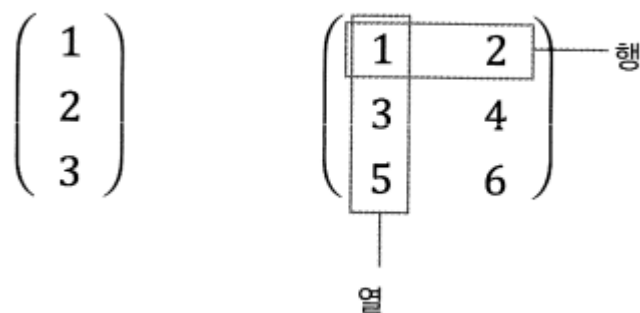
크기와 방향을 가진 양이다.

숫자가 일렬로 늘어선 집합으로 표현이 가능하고 1차원 배열로 취급 가능

행렬이란?

숫자가 2차원 형태(사각형의 형상)로 늘어진 것이다.

그림 1-1 벡터와 행렬의 예



벡터를 표현하는 방법은

1. 세로로 나열 (열벡터)
2. 가로로 나열 (행벡터)

→ 이 책에서는 행벡터가 구현할때 편리하기 때문에 행벡터로 다룰 예정이다.

행렬을 취급할 때 단골 라이브러리는 "넘파이" 이다!

```
[1] import numpy as np
```

```
[2] x = np.array([1,2,3])
```

```
[3] x.__class__ #속해있는 class
```

```
⇒ numpy.ndarray
```

```
[4] x.shape #모양
```

```
⇒ (3,)
```

```
[6] x.ndim #차원
```

```
⇒ 1
```

```
[7] w = np.array([[1,2,3],[4,5,6]])
```

```
[8] w.shape
```

```
⇒ (2, 3)
```

```
[9] w.ndim
```

```
⇒ 2
```

1.1.2. 행렬의 원소별 연산

```
# 행렬의 원소별 연산
```

```
W = np.array([[1,2,3],[4,5,6]])
```

```
X = np.array([[0,1,2],[3,4,5]])
```

```
W+X
```

```
array([[ 1,  3,  5],  
       [ 7,  9, 11]])
```

```
W*X
```

```
array([[ 0,  2,  6],  
       [12, 20, 30]])
```

1.1.3. 브로드캐스트

```
A = np.array([[1,2],[3,4]])
```

```
A*10
```

```
array([[10, 20],  
       [30, 40]])
```

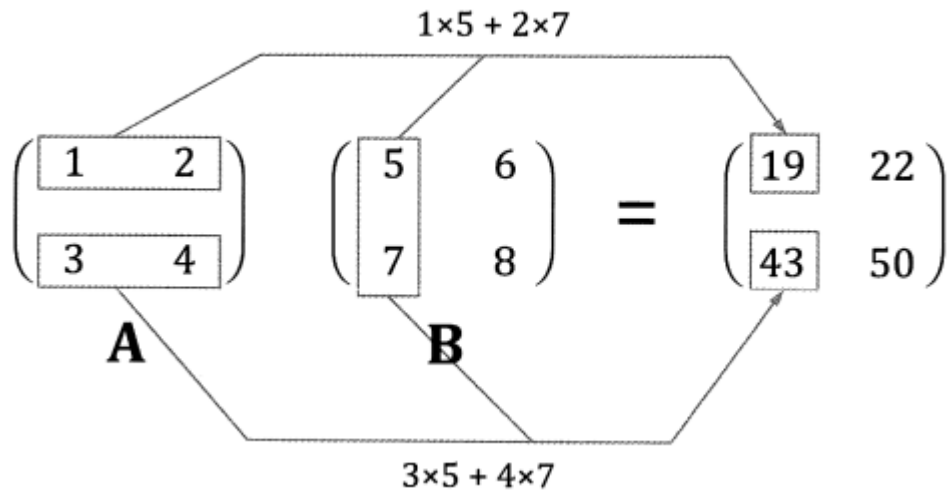
```
A = np.array([[1,2],[3,4]])  
b = np.array([10,20])  
A*b
```

```
array([[10, 40],  
       [30, 80]])
```

⇒ 넘파이의 다차원 배열에서는 형상이 다른 배열끼리도 연산이 가능하다

1.1.4. 벡터의 내적과 행렬의 곱

그림 1-5 행렬의 곱셈 방법



#벡터의 내적과 행렬 곱

```
a = np.array([1,2,3])  
b = np.array([4,5,6])
```

```
np.dot(a,b)
```

```
np.int64(32)
```

```
A = np.array([[1,2],[3,4]])
```

```
B = np.array([[5,6],[7,8]])
```

```
np.matmul(A,B)
```

```
array([[19, 22],  
       [43, 50]])
```

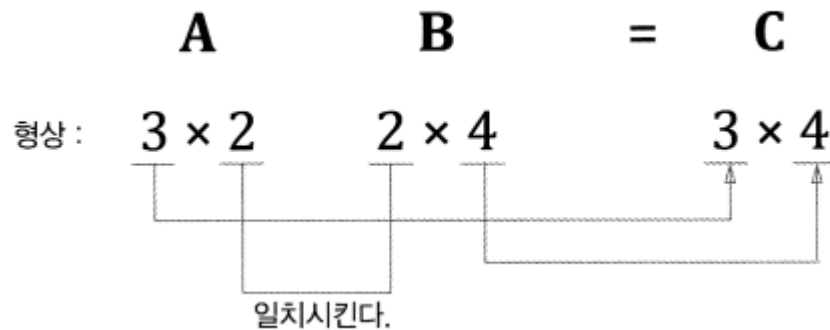
⇒ 행렬의 계산을 도와주는 메서드

→ np.dot()

→ np.matmul()

1.1.5. 행렬 형상 확인

그림 1-6 형상 확인: 행렬의 곱에서는 대응하는 차원의 원소 수를 일치시킨다.



⇒ C의 행렬 형상 확인하기

1.2. 신경망의 추론

신경망에서 수행하는 작업

1. 학습
2. 추론

1st 추론

1.2.1. 신경망 추론 전체 그림

신경망은 간단히 말해 단순한 "함수"

함수란?

무언가를 입력하면 무언가를 출력해주는 변환기

→ **신경망도 함수처럼 입력을 출력으로 변환한다.**

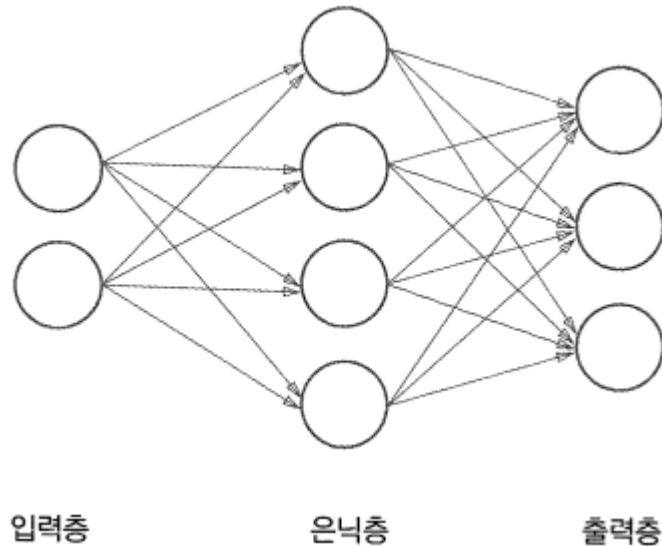
2차원 data를 입력해서 3차원 data로 출력하려면?

⇒ 신경망으로 구현해보자

⇒ 입력층에 뉴런 2개 → 출력층에 뉴런 3개 ready

→ 은닉층(=중간층)에는? 적당한 수의 뉴런을 배치

그림 1-7 신경망의 예



⇒ 이때 화살표에 가중치가 존재한다.

⇒ 가중치와 뉴런의 값을 각각 곱하여 그 합이 다음 뉴런의 입력으로 쓰임

→ 사실 그 합에다 "활성화 함수 (activation function)를 적용한 값이 다음 값으로 입력

→ 추가적으로 각 층에서 이전 뉴런의 값에 영향 받지 않는 "정수"도 더해진다

→ 이 정수는 "편향(bias)" 이라고 한다.

→ 인접하는 층의 모든 뉴런과 연결되어 있다는 뜻에서

→ 이러한 1-7같이 인접하는 층의 모든 뉴런과 연결로 되어있다면

→ **완전연결계층** 이라고 부른다.

→ 수식으로 나타내면

$$h_1 = x_1 w_{11} + x_2 w_{21} + b_1$$

가중치와 편향에는 인덱스(=첨자)가 붙는다

x11, x12 등등...

규칙은 중요하지 않으나, 이게 "가중치 합"으로 계산된다는 것이 중요하다.

실제로 **완전 연결계층이 수행하는 변환** → **행렬의 곱**을 이용하여 정리해보자.

$$(h_1, h_2, h_3, h_4) = (x_1, x_2) \begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \end{pmatrix} + (b_1, b_2, b_3, b_4)$$

h는 은닉층들의 뉴런을 뜻한다

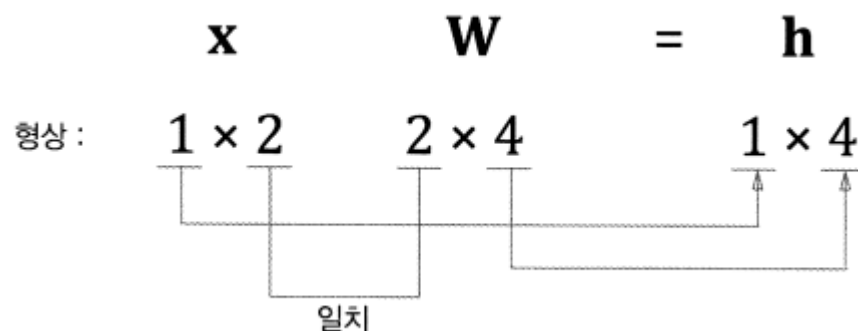
간소화 한다면?

$$\rightarrow \mathbf{h} = \mathbf{xW} + \mathbf{b}$$

이렇게 나타낼 수 있다.

여기에서도 대응하는 차원의 수가 일치해야 한다! (편향은 관계 없음)

그림 1-8 형상 확인: 대응하는 차원의 원소 수가 일치함(편향은 생략)



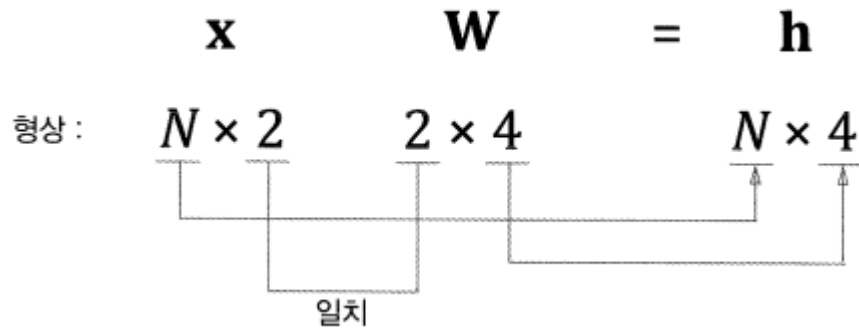
지금까지 한 건 하나의 샘플 데이터(입력 데이터)만 대상으로 했는데

신경망의 추론이나 학습에서는 **다수의 샘플 데이터 (미니배치)**를 한 번에 처리한다.

→ 행렬의 x 행 각각에 샘플 data를 하나씩 저장해야한다.

N개의 샘플 데이터를 미니 배치로 한꺼번에 처리?

그림 1-9 형상 확인: 미니배치 버전의 행렬 곱(편향은 생략)



```
[28] #미니배치 버전 파이썬으로 구현하기
```

```
[29] W1 = np.random.randn(2,4) #가중치
```

```
[30] b1 = np.random.randn(4) #편향
```

```
[31] x = np.random.randn(10,2) #입력
```

```
[32] h = np.matmul(x,W1) + b1
```

```
▶ print(h)
```

```
[[ 2.05296688e+00 -3.41384602e-01 -5.65969378e-02 -5.60658782e-01]
 [ 1.01224704e+00 -6.93310906e-01 -7.38829963e-01  5.54011689e-01]
 [ 1.87909354e+00 -7.16901767e-01  1.52146416e+00  6.89696080e-01]
 [-8.21932667e-01 -1.24099570e+00 -2.32882474e+00  2.27475015e+00]
 [ 3.49889526e+00 -4.81299883e-01  4.25090374e+00  3.54796242e-03]
 [ 2.10196160e+00 -6.27280049e-01  1.59139452e+00  4.03089232e-01]
 [ 1.23970452e+00 -7.29645214e-01  1.53047148e-02  6.90893889e-01]
 [ 4.03481046e+00 -3.22803301e-01  4.72363194e+00 -4.94090144e-01]
 [ 5.05090106e-01 -8.00860265e-01 -1.41293176e+00  8.82347873e-01]
 [ 2.16658661e+00 -8.30333539e-01  2.83529629e+00  1.08952007e+00]]
```

→ 10개의 샘플 데이터 각각을 완전연결계층으로 변환함

x의 첫 번째 차원은 각 샘플 데이터에 해당한다.

x[0] : 0번째 입력 데이터

x[1] : 1번째 입력 데이터

이런식으로 뉴런이 저장된 것이다

b1인 편향을 덧셈할때는 브로드캐스트 된다!

b1의 형상은 (4,)인 상태인데

더해질 때 자동으로 (10,4)로 복제되어 10개로 바뀐다!

완전연결계층에 의한 변환은 "선형"변환이다

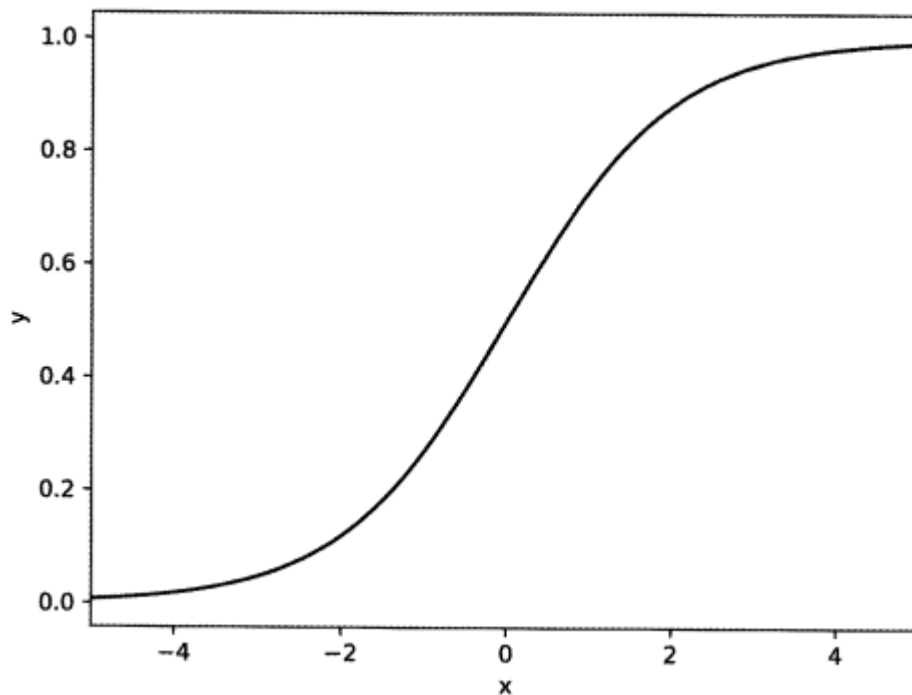
여기에 "비선형" 효과를 부여 ⇒ **활성화 함수**

→ 이 비선형 활성화 함수를 이용하면 신경망의 표현력을 높일 수 있다

→ 활성화 함수는 매~우 다양하지만 여기선 **시그모이드 함수**를 사용해보겠다.

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

그림 1-10 시그모이드 함수의 그래프



```
import numpy as np

#식을 코드로 변환
def sigmoid(x):
    return 1/(1+np.exp(-x))
```

```
x = np.random.randn(10,2)
W1 = np.random.randn(2,4)
b1 = np.random.randn(4)
W2 = np.random.randn(4,3)
b2 = np.random.randn(3)
```

```
h = np.matmul(x,W1) + b1
a = sigmoid(h)
s = np.matmul(a,W2) + b2
```

* (10,3)은 10개의 데이터가 한꺼번에 처리되며 각 데이터가 3차원의 데이터로 변환되었다는 뜻이다

np.random.randn(10, 2)이란?

- np.random.randn 는 평균이 0이고 표준편차가 1인 **표준 정규분포**(Standard Normal Distribution, $N(0,1)$)에서 난수를 뽑습니다.
- 10, 2 는 **10행 2열의 배열**을 만들겠다는 뜻입니다.

**

```
array([[ 0.123, -1.456],
       [ 0.789,  0.321],
```

...

```
[10개 행, 각 행에 2개 값]])
```

⇒ 이러한 형태의 2차원 배열을 반환한다

**

요약:

- 값 분포: 평균 0, 표준편차 1
- 형태: 10x2 배열
- 사용 예: 머신러닝에서 더미 데이터 만들 때 자주 씀

각 차원은 각 클래스에 대응하는 "점수"가 된다.

그래서 차원의 값을 이용하여 3 클래스 분류를 할 수 있다.

실제로 분류를 한다면 출력층에서 가장 큰 값을 내뱉는 뉴런에 해당하는 클래스가 예측 결과가 된다.

참고로

점수는 "확률"이 되기 전의 값이라

점수가 높을수록 뉴런에 해당하는 클래스의 확률이 높아진다는 것이다.

이 점수를 소프트맥스 함수를 입력하면 확률을 얻을 수 있다.

1.2.2. 계층으로 클래스화 및 순전파 구현

완전연결계층에 의한 변환을 Affine(아핀)계층

시그모이드 함수에 의한 변환을 Sigmoid 계층

으로 구현하겠다

완전연결계층에 의한 변환은 아핀 변환에 해당 (안 중요)

각 계층은 파이썬 클래스로 구현, 기본 변환을 수행하는 메서드의 이름은 forward()

신경망 추론 과정에서 하는 처리는?

⇒ **신경망의 순전파 (forward propagation)**

→ 말 그대로 입력층에서 출력층으로 향하는 전파

→ 입력부터 출력까지 처리 결과를 차례대로 전파한다.

참고로 반대방향으로 데이터를 전파하는 것을

⇒ **신경망의 역전파 (backward propagation)**라고 한다.

params는 가중치 & 편향 (매개변수)을 담은 리스트이다

grads는 params에 저장된 각 매개변수에 대응, 매개변수의 기울기를 보관하는 리스트

→ 앞으로 순전파만 구현할 것이라 2가지의 규칙을 적용하겠다

1. 각 계층은 forward 메서드만 가진다
2. 매개변수들은 params 인스턴스 변수에 보관한다.

Sigmoid 계층 구현하기

```
[40] class Sigmoid:
      def __init__(self):
          self.params = []

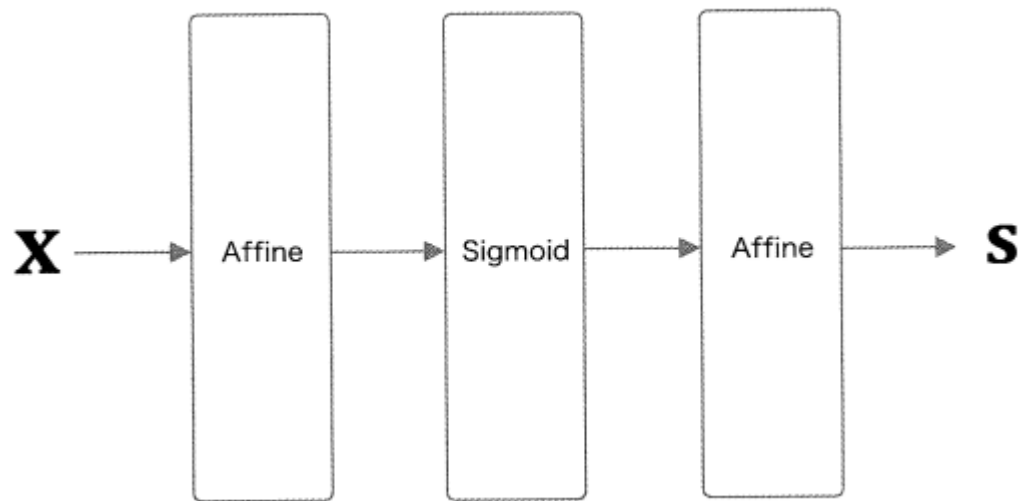
      def forward(self, x):
          return 1/(1+np.exp(-x))
```

```
[41] class Affine:
      #시그모이드 계층에는 학습하는 매개변수가 따로 없어서 초기화 (비어있도록)
      def __init__(self,W,b):
          self.params = [W,b]

      def forward(self,x):
          W,b = self.params
          out = np.matmul(x,W)+b
          return out
```

그렇다면 Sigmoid를 활용하여 아래와 같은 신경망을 구현해 보자

그림 1-11 구현해볼 신경망의 계층 구성



점수인 S 를 출력하게 되는데

이 신경망을 TwoLayerNet이라는 클래스로 추상화하고

주 추론 처리는 `predict(x)` 메서드로 구현

`__init__` : 초기화 메서드

⇒ 가중치를 초기화하고 3개의 계층을 생성한다. (Affine, Sigmoid, Affine)

⇒ 마지막으로는 학습해야 할 가중치 매개변수들을 `params` 리스트에 저장

⇒ 모든 계층은 학습 매개변수들을 인스턴스 변수인 `params`에 보관중

```

class TwoLayerNet:
    def __init__(self, input_size, hidden_size, output_size):
        I, H, O = input_size, hidden_size, output_size

        #가중치와 편향 초기화
        W1 = np.random.randn(I,H)
        b1 = np.random.randn(H)
        W2 = np.random.randn(H,O)
        b2 = np.random.randn(O)

        #계층 생성
        self.layers = [
            Affine(W1, b1),
            Sigmoid(),
            Affine(W2, b2)
        ]

        #모든 가중치를 리스트에 모은다
        self.params = []
        for layer in self.layers:
            self.params += layer.params

    def predict(self, x):
        for layer in self.layers:
            x = layer.forward(x)

        return x

```

매개변수들을 하나의 리스트에 보관하면 “매개변수 갱신”과 “매개변수 저장”을 손쉽게 처리할 수 있기 때문에 이처럼 하나의 리스트에 모은다.

plus) 리스트 결합 연산은 +를 이용하여 이루어진다.

#파이썬에서 + 연산자는 리스트들을 결합해준다.

```
a = ['A', 'B']  
a = a+['C', 'D']
```

a

['A', 'B', 'C', 'D']

TwoLayerNet 구현에서는 각 계층의 params 리스트를 더해주는 것 만으로도 모든 학습 매개변수를 하나의 리스트에 담은 것임!

1.3. 신경망의 학습

"학습"된 신경망만이 "좋은 추론"을 할 수 있다.

So, 학습을 먼저 수행하고 학습된 매개변수를 이용해 추론을 수행하는 것이 일반적이다.

추론이란?

다중 class 분류 등의 문제에 답을 구하는 작업

신경망의 학습이란?

최적의 매개변수 값을 찾는 작업

1.3.1. 손실함수

학습이 얼마나 잘 되고 있는지를 알기 위해 "척도"가 필요

So, 성능을 나타내는 척도로 **손실(loss)**을 사용한다

손실이란, 학습 데이터와 신경망 vs 예측 결과

→ 예측이 얼마나 나쁜지를 산출한 값

그래서 신경망의 손실은 "**손실함수**"를 사용하여 구함

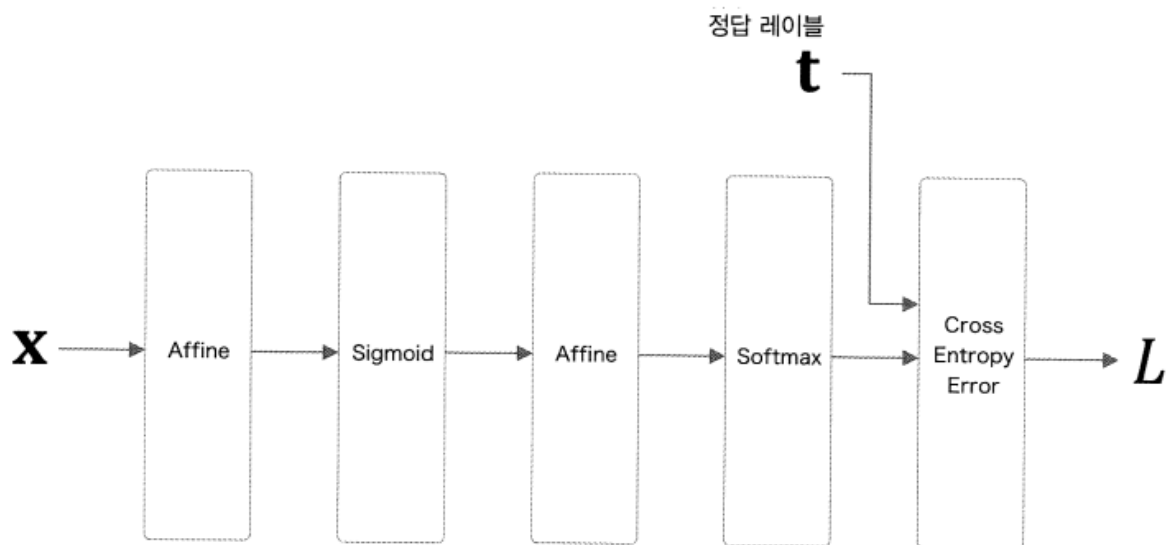
다중 클래스 분류 신경망에서는 "손실함수"로 보통 교차 엔트로피 오차를 이용한다

⇒ 신경망이 출력하는 각 클래스의 "확률"과 "정답 레이블"을 이용하여 구할 수 있다.

Softmax계층과 Cross Entropy Error 계층을 새로 추가한다.

이 신경망의 구성을 "계층 관점"에서 그리면 다음과 같다.

그림 1-12 손실 함수를 적용한 신경망의 계층 구성



x: 입력 데이터

t: 정답 레이블

L: 손실

Softmax 계층의 출력은 확률이 되고,

다음 계층인 Cross Entropy Error 계층에

1. 확률
2. 정답레이블

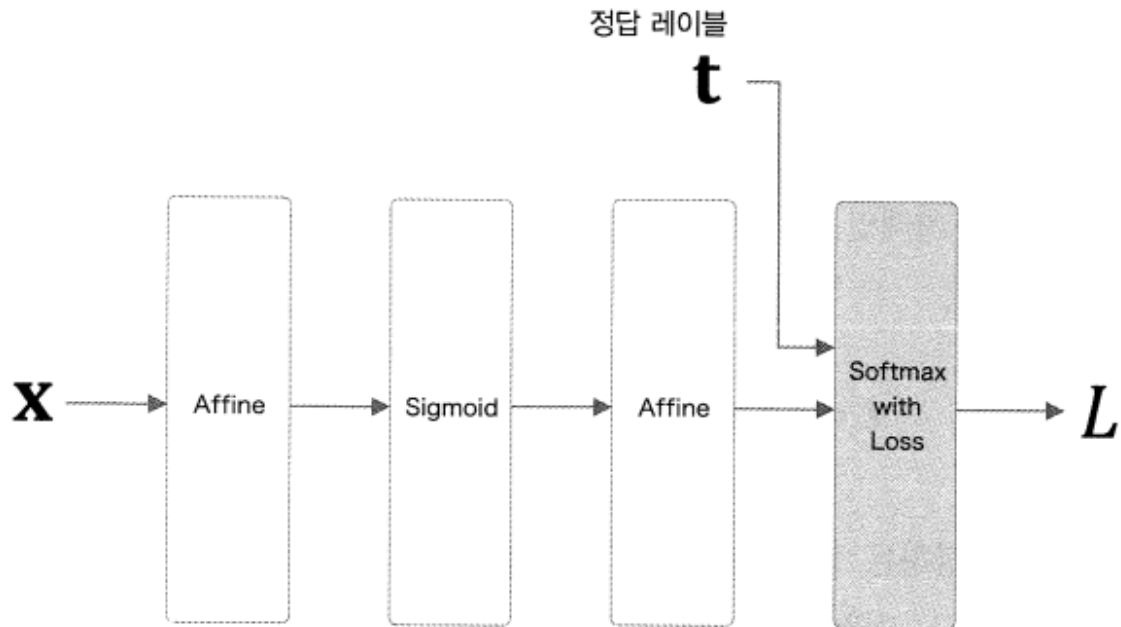
이 2가지를 입력한다.

소프트맥스 함수와 교차 엔트로피 오차를 계산하는 계층을

“Softmax with Loss” 계층 하나로 구현할 것임

Why? 두개의 계층을 통합하면 역전파 계산이 쉬워지기 때문

그림 1-13 Softmax with Loss 계층을 이용하여 손실을 출력한다.



1.3.2. 미분과 기울기

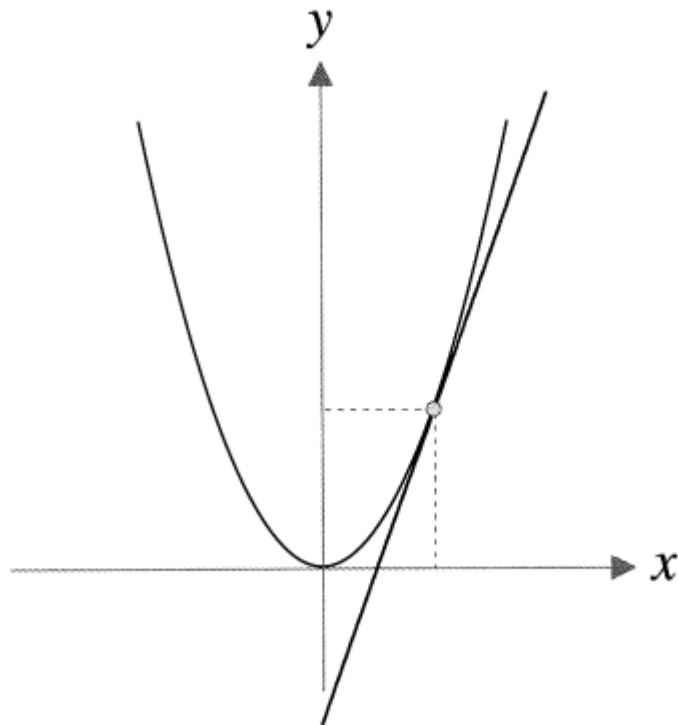
신경망 학습의 목표:

손실을 최소화하는 매개변수를 찾는 것

⇒ 이때 중요한 것은 **“미분” & “기울기”**

** x 를 조금 “변화”시켰을 때 y 의 값이 얼마나 변하는지 “변화의 정도”를 알 수 있는 것이 dy/dx

그림 1-14 $y = x^2$ 의 미분은 각 x 에서의 기울기를 나타낸다.



그럼 이 식에서 변화의 정도는?

→ $dy/dx = 2x$ 이다

→ 이것은 함수의 "기울기"에 해당한다.

행렬에서도 함수의 기울기를 구할 수 있다.

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{pmatrix} \frac{\partial L}{\partial W_{11}} & \cdots & \frac{\partial L}{\partial W_{1n}} \\ \vdots & \ddots & \\ \frac{\partial L}{\partial W_{m1}} & & \frac{\partial L}{\partial W_{mn}} \end{pmatrix}$$

1.3.3. 연쇄 법칙

학습 시 신경망은 학습 데이터를 주면 손실을 출력한다

여기서 우리가 얻고 싶은 것은? ⇒ 각 매개변수에 대한 손실의 기울기

기울기를 얻을 수 있다면 기울기를 사용해 매개변수를 갱신할 수 있음

그럼 신경망의 기울기는 어떻게 구할까?

⇒ “오차역전파법”

⇒ “연쇄 법칙”

: 합성함수에 대한 미분의 법칙 (최종출력 z 에 대해)

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}$$

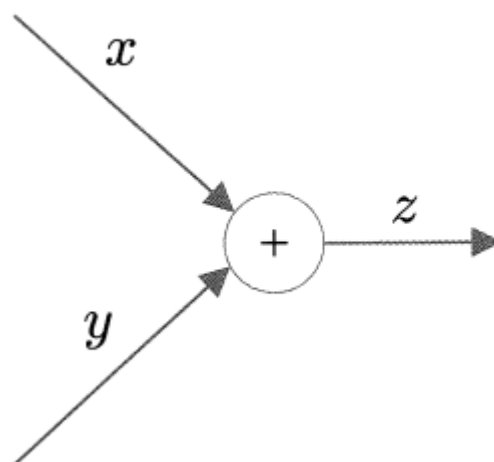
연쇄 법칙이 중요한 이유는 아무리 복잡해도 (아무리 많은 함수를 연결해도)

미분은 개별 함수의 미분들을 이용해서 구할 수 있기 때문

⇒ 함수의 국소적인 미분을 계산할 수 있다면, 그 값들을 곱해서 전체의 미분을 구할 수 있다.

1.3.4. 계산 그래프

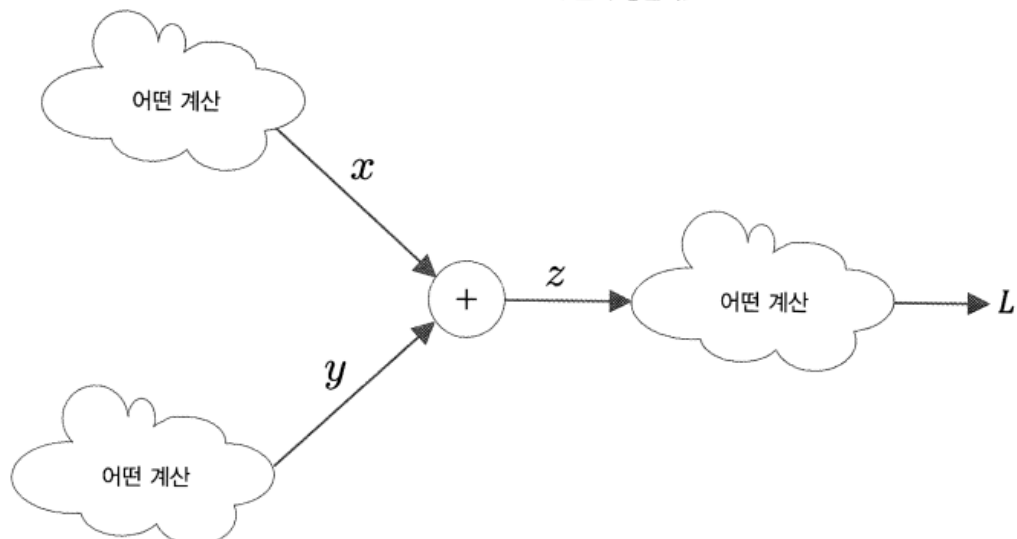
그림 1-15 $z = x + y$ 를 나타내는 계산 그래프



더하기를 '+' 노드로 나타내었다.

참고로 이렇게 처리 결과 순서대로 흐르는 것이 순전파이다.

그림 1-16 앞뒤로 추가된 노드는 '복잡한 전체 계산'의 일부를 구성한다.

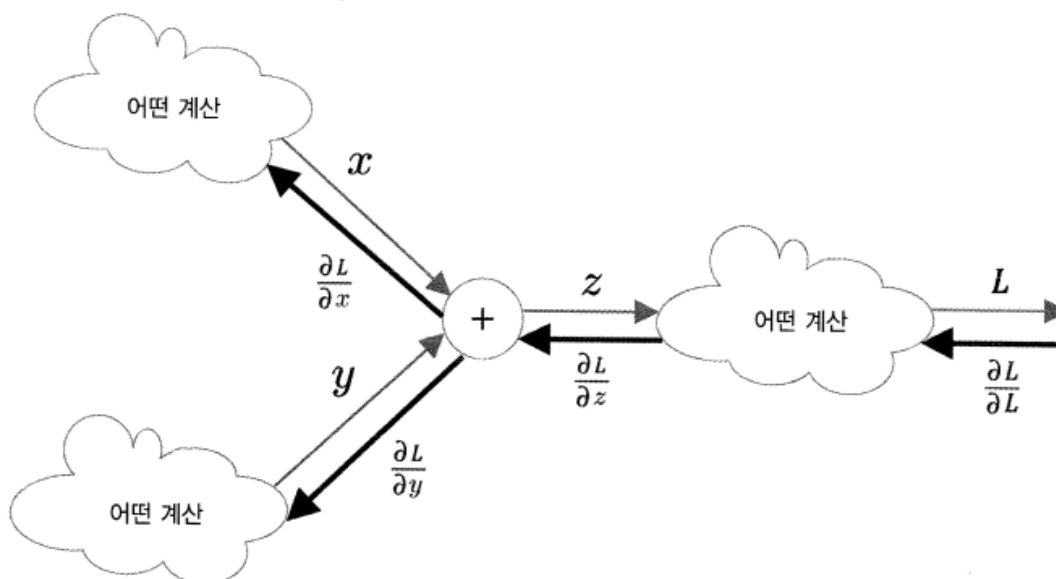


⇒ L은 loss 값인 손실이다 (해당 값은 스칼라로 출력된다)

그럼 역전파는?

다음과 같다

그림 1-17 계산 그래프의 역전파



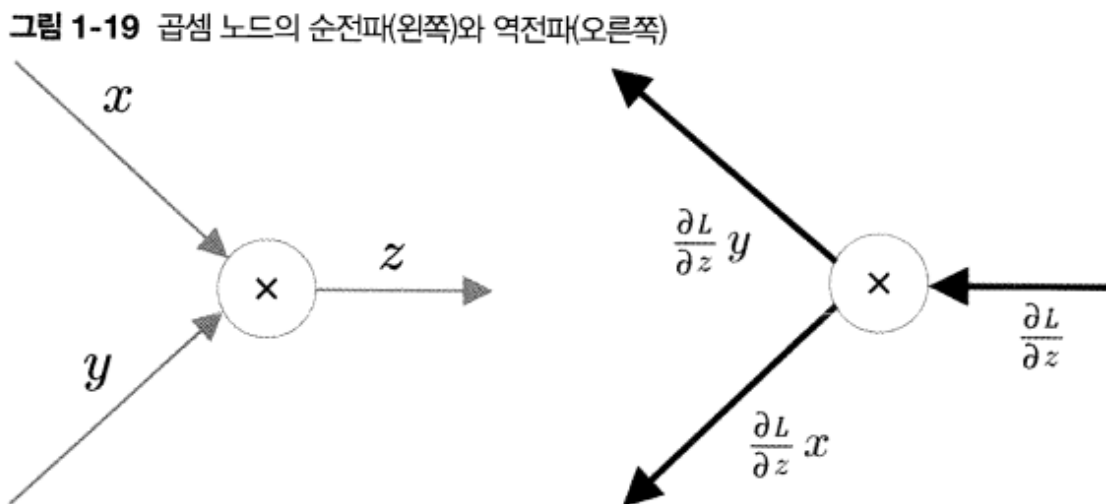
⇒ 보통 역전파는 두꺼운(붉은) 화살표로 그리고 그 밑에 전파되는 값을 쓴다.

⇒ “전파되는 값”은 최종 출력 L 의 각 변수에 대한 미분이다.

연산 노드로는 “덧셈 노드” 외에도 다양한 연산이 있다.

▶ 곱셈 노드

상류로부터 받은 기울기에 순전파 시의 입력을 서로 바꾼 값을 곱한다.



참고로 다변수를 흘려도 문제 없다 (ex] 벡터나 행렬, 다변수 등등)

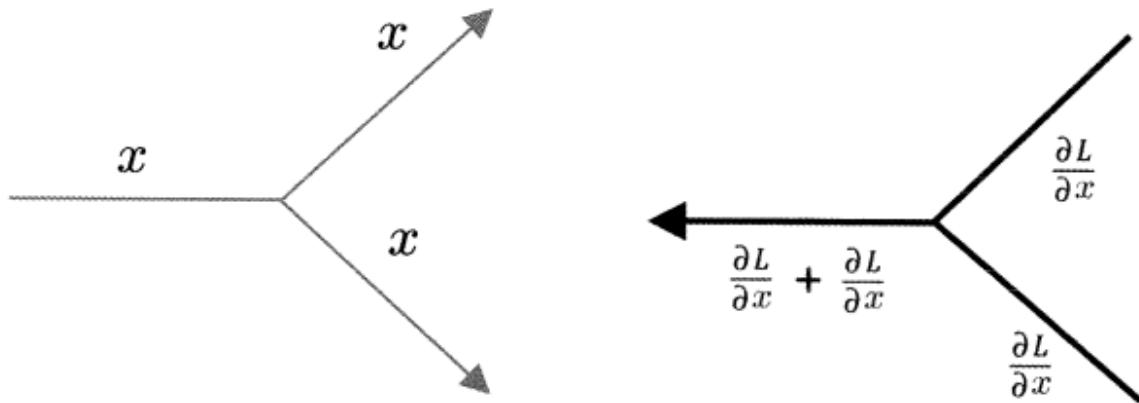
▶ 분기 노드

그냥 선이 두개로 나뉘지도록 그리는 것

이때 같은 값이 복제되어 분기함 ⇒ “복제 노드” 라고 할 수 있음

그리고 이때의 역전파는 상류에서 온 기울기들의 합이 된다.

그림 1-20 분기 노드의 순전파(왼쪽)와 역전파(오른쪽)

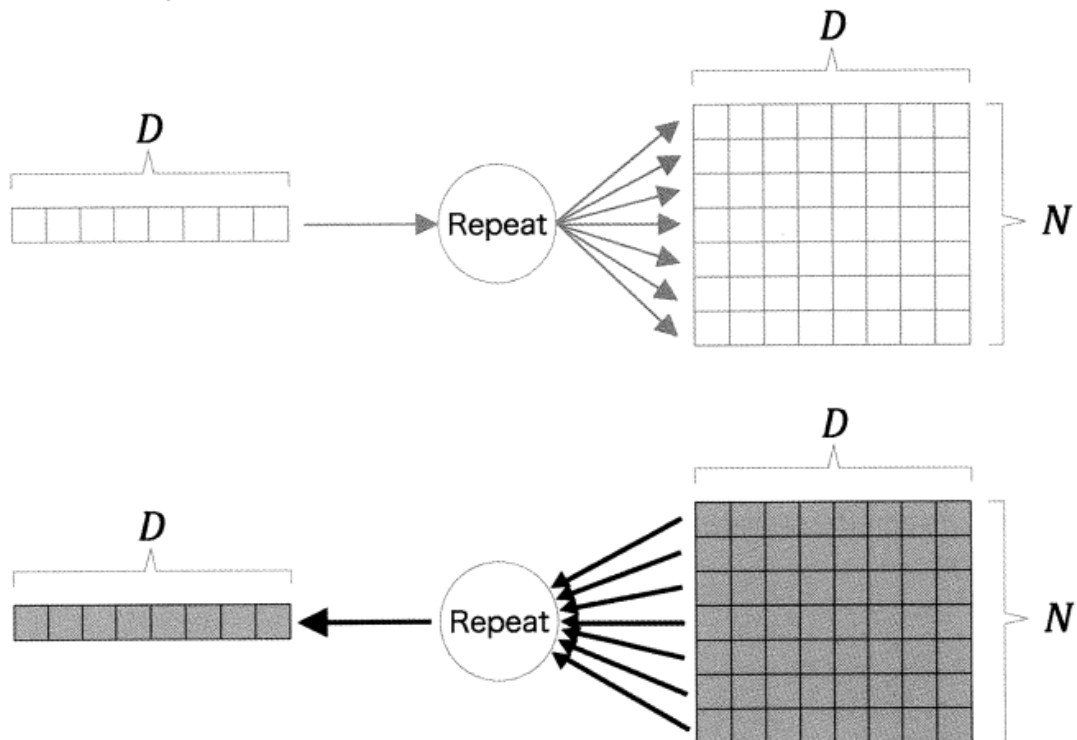


▶ Repeat 노드

2개로 분기하는 분기 노드를 일반화하면 N개의 분기 (복제)가 된다.

이걸 "Repeat 노드"라고 부른다

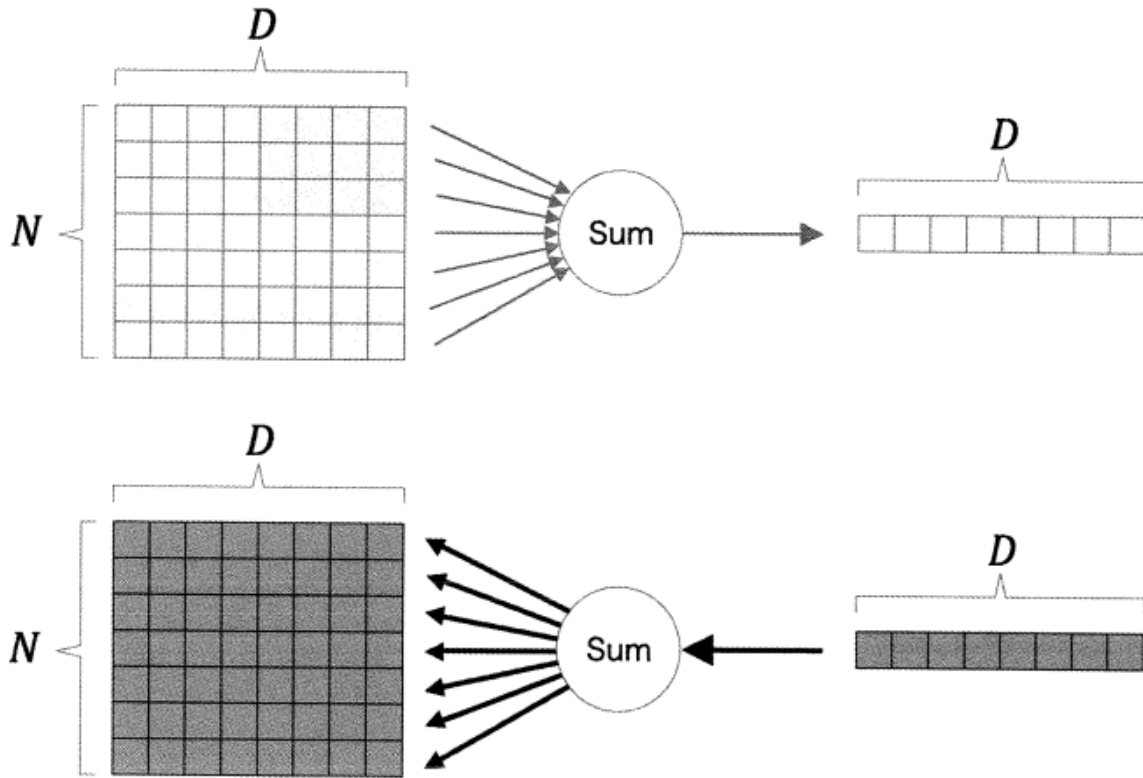
그림 1-21 Repeat 노드의 순전파(위)와 역전파(아래)



▶ Sum 노드

범용 덧셈 노드

그림 1-22 Sum 노드의 순전파(위)와 역전파(아래)

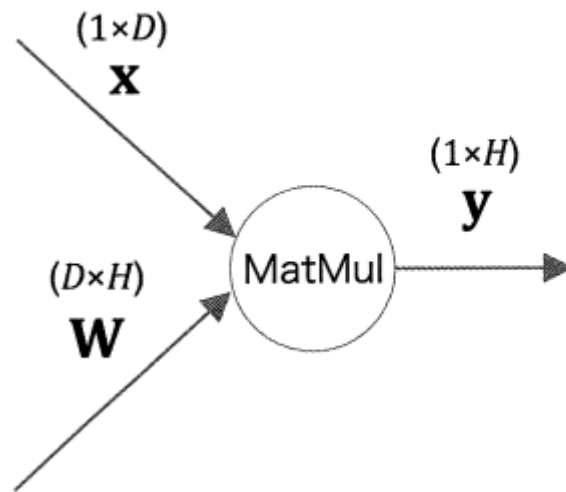


역전파가 `np.repeat()` 메서드로 구현할 수 있는데
그래서 Sum노드와 Repeat 노드는 서로 “반대 관계” 라고 불린다.

▶ MatMul 노드

⇒ 행렬의 곱셈

그림 1-23 MatMul 노드의 순전파: 각 변수 위에 형상을 표시함



$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \mathbf{W}^T$$

이때 $\mathbf{W}^T$ 는 전치 행렬이라는 뜻으로 최종적으로 형상 확인을 했을 땐 다음과 같다

그림 1-24 행렬 곱의 형상 확인

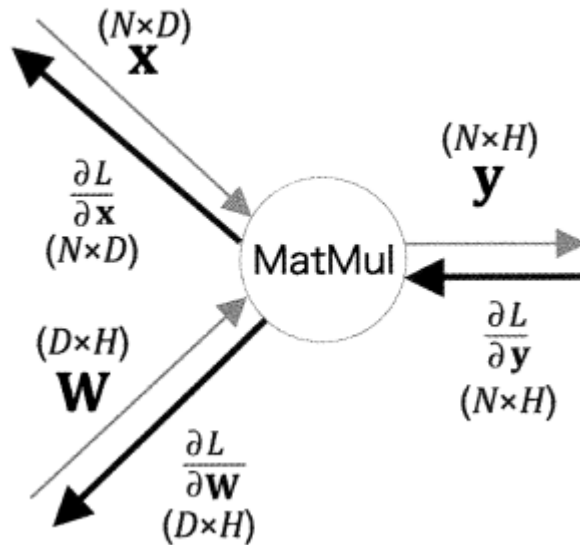
$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \mathbf{W}^T$$

형상: $1 \times D \quad 1 \times H \quad H \times D$

The diagram shows the shapes of the terms in the equation. Below the equation, the shapes are listed: $1 \times D$ for $\frac{\partial L}{\partial \mathbf{x}}$, $1 \times H$ for $\frac{\partial L}{\partial \mathbf{y}}$, and $H \times D$ for $\mathbf{W}^T$. Arrows indicate the flow of shapes: from $1 \times D$ and $1 \times H$ to the first $1 \times D$, and from $1 \times H$ and $H \times D$ to the $1 \times H$.

그래서 역전파로 나타내게 되면

그림 1-25 MatMul 노드의 역전파



핵심 요약:

- **기능:** 두 개의 텐서를 곱하는 행렬 연산을 수행함
- **입력 조건:**
 - 일반적으로 2차원 이상의 텐서를 입력으로 받음
 - 차원이 맞아야 곱셈이 가능 ($A @ B$ 에서 A 의 마지막 차원 = B 의 앞에서 두 번째 차원)
- **브로드캐스팅 지원:** 차원이 맞지 않아도 자동으로 확장해 곱할 수 있음 (ONNX 등에서 지원)
- **활용 예시:** 완전연결층 ($Affine$ 또는 $Linear$), 어텐션, 트랜스포머 등에서 핵심 역할

딥러닝에서는 보통

$Y = X @ W + b$ 에서 $X @ W$ 부분이 $MatMul$ 에 해당
입력 X 에 가중치 W 를 곱하는 부분임

1.3.5. 기울기 도출과 역전파 구현

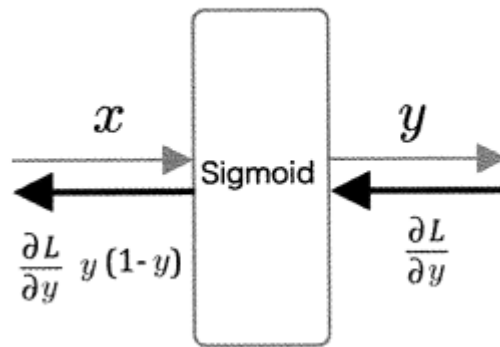
▶ Sigmoid 계층

$$y = \frac{1}{1 + \exp(-x)}$$

위를 미분하면

$$\frac{\partial y}{\partial x} = y(1 - y)$$

그림 1-28 Sigmoid 계층의 계산 그래프



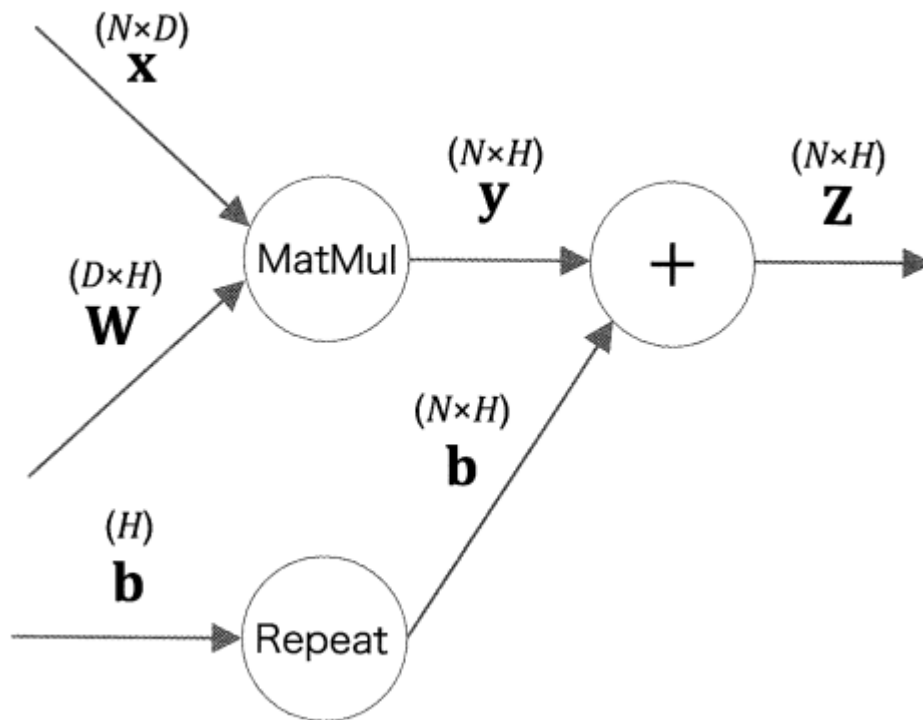
▶Affine(어파인) 계층

계층의 순전파는

$y = \text{np.matmul}(x, W) + b$ 로 구현 가능하다

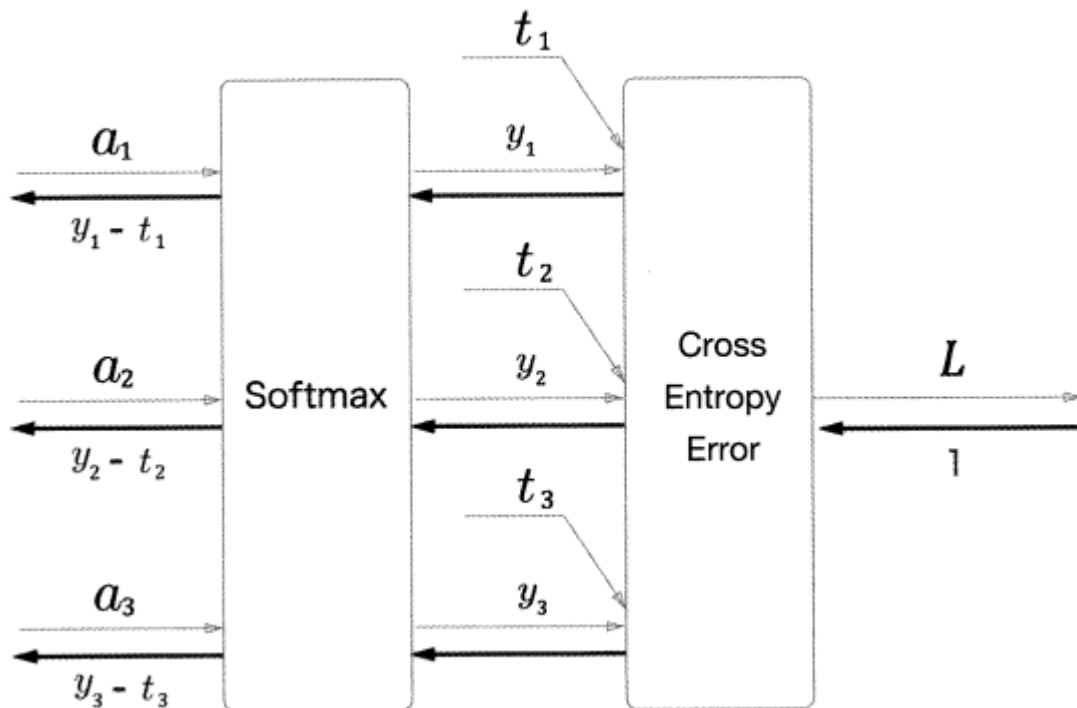
b 는 넘파이의 브로드캐스트 사용

그림 1-29 Affine 계층의 계산 그래프



► Softmax with Loss 계층

그림 1-30 Softmax with Loss 계층의 계산 그래프



소프트맥스 함수는 Softmax 계층이고, 교차 엔트로피 오차는 Cross Entropy Error 계층이다.

1.3.6. 가중치 갱신

1: 미니배치

→ 훈련 데이터 중에서 무작위로 다수의 데이터 골라내기

2: 기울기 계산

→ 오차역전파법으로 각 가중치 매개변수에 대한 손실 함수의 기울기 구하기

3: 매개변수 갱신

기울기를 사용하여 가중치 매개변수를 갱신

4: 반복

1~3단계를 필요한 만큼 반복한다.

매개변수는 기울기와 반대 방향으로 갱신하면 손실을 줄일 수 있는데 이것
“경사하강법”이라고 부른다.

3단계에서 수행하는 가중치 갱신 기법의 종류 中 가장 단순한 것

: 확률적경사하강법(SGD)

** 확률적이라는 말은 무작위로 선택된 데이터에 대한 기울기를 이용한다는 뜻

SGD는 현재의 가중치를 기울기 방향으로 일정한 거리만큼 갱신한다.

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}$$

1.4. 신경망으로 문제 풀기

간단한 데이터 셋으로 신경망 학습시키기

1.4.1. 스파이럴 데이터셋

```
import sys
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/conte

```
import os
os.chdir('drive/MyDrive/DL_BAMBOO/')
sys.path.append('drive/MyDrive/DL_BAMBOO/')
```

```
import spiral
```

```
import matplotlib.pyplot as plt
```

```
x,t = spiral.load_data()
```

```
print('x',x.shape)
```

```
x (300, 2)
```

```
print('t', t.shape)
```

```
t (300, 3)
```

1.4.2. 신경망 구현

✓ [49] #신경망 구현
0초

✓ [50] import sys
0초

✓ [54] import sys
import numpy as np
from layers import Affine, Sigmoid, SoftmaxWithLoss
0초

```
class TwoLayerNet:
    def __init__(self, input_size, hidden_size, output_size):
        I, H, O = input_size, hidden_size, output_size

        # 가중치와 편향 초기화
        W1 = 0.01 * np.random.randn(I, H)
        b1 = np.zeros(H)
        W2 = 0.01 * np.random.randn(H, O)
        b2 = np.zeros(O)

        # 레이어 생성
        self.layers = [
            Affine(W1, b1),
            Sigmoid(),
            Affine(W2, b2)
        ]
        self.loss_layer = SoftmaxWithLoss()

        # 모든 가중치와 기울기를 리스트에 모은다.
        self.params, self.grads = [], []
        for layer in self.layers:
            self.params += layer.params
            self.grads += layer.grads

    def predict(self, x):
        for layer in self.layers:
            x = layer.forward(x)
        return x

    def forward(self, x, t):
        score = self.predict(x)
        loss = self.loss_layer.forward(score, t)
        return loss

    def backward(self, dout=1):
        dout = self.loss_layer.backward(dout)
        for layer in reversed(self.layers):
            dout = layer.backward(dout)
        return dout
```

1.4.3. 학습용 코드

✓
5초

```
[57] import matplotlib
import matplotlib.pyplot as plt
from tqdm import tqdm # pip install tqdm
from optimizer import SGD
import spiral
#import TwoLayerNet

matplotlib.rc('font', family='Malgun Gothic') # linux
# matplotlib.rc('font', family='AppleGothic') # Mac

# 1. 하이퍼파라미터 설정
max_epoch = 300
batch_size = 30
hidden_size = 10
learning_rate = 1.0

# 2. 데이터 읽기, 모델과 옵티마이저 생성
x, t = spiral.load_data()
model = TwoLayerNet(input_size=2,
                    hidden_size=hidden_size,
                    output_size=3)
optimizer = SGD(lr=learning_rate)

# 학습에 사용하는 변수
data_size = len(x)
max_iters = data_size // batch_size
total_loss = 0
loss_count = 0
loss_list = []

for epoch in tqdm(range(max_epoch)):
    # 3. 데이터 셔플링
    idx = np.random.permutation(data_size)
    x = x[idx]
    t = t[idx]

    for iters in range(max_iters):
        batch_x = x[iters*batch_size:(iters+1)*batch_size]
        batch_t = t[iters*batch_size:(iters+1)*batch_size]

        # 기울기를 구해 매개변수 갱신
        loss = model.forward(batch_x, batch_t)
        model.backward()
        optimizer.update(model.params, model.grads)

        total_loss += loss
        loss_count += 1

    # 정기적으로 학습 경과 출력
    if (iters+1) % 10 == 0:
        avg_loss = total_loss / loss_count
        print(f'| 에폭 {epoch+1} | 반복{iters+1}/{max_iters} | 손실 {avg_loss:.2f}')
        loss_list.append(avg_loss)
        total_loss, loss_count = 0, 0
```

```

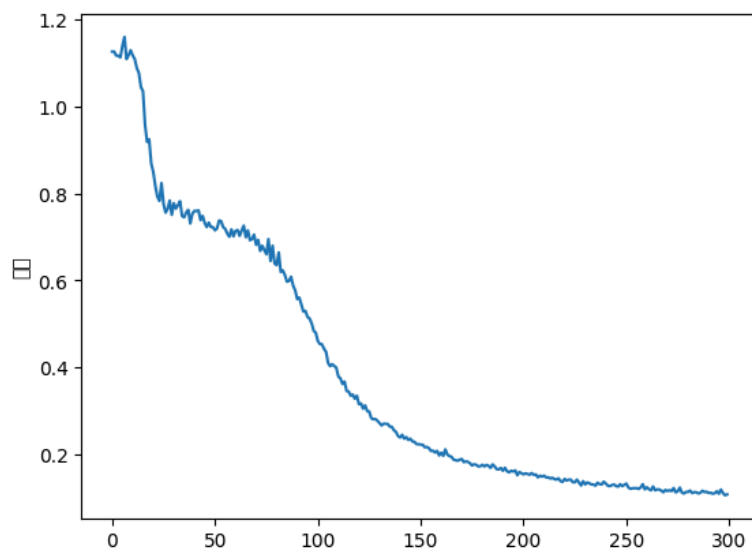
# 학습 결과 플롯
plt.plot(np.arange(len(loss_list)), loss_list, label='train')
plt.xlabel('반복 (x10)')
plt.ylabel('손실')
plt.show()

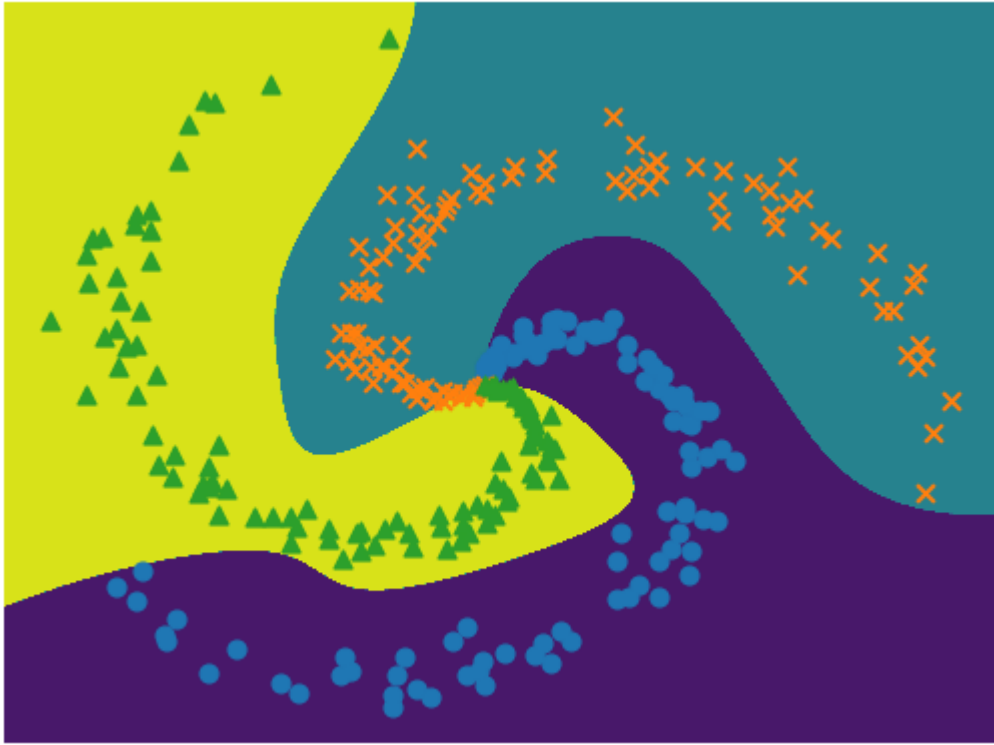
# 경계 영역 플롯
h = 0.001
x_min, x_max = x[:, 0].min() - .1, x[:, 0].max() + .1
y_min, y_max = x[:, 1].min() - .1, x[:, 1].max() + .1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
X = np.c_[xx.ravel(), yy.ravel()]
score = model.predict(X)
predict_cls = np.argmax(score, axis=1)
Z = predict_cls.reshape(xx.shape)
plt.contourf(xx, yy, Z)
plt.axis('off')

# 데이터점 플롯
x, t = spiral.load_data()
N = 100
CLS_NUM = 3
markers = ['o', 'x', '^']
for i in range(CLS_NUM):
    plt.scatter(x[i*N:(i+1)*N, 0], x[i*N:(i+1)*N, 1], s=40, marker=markers[i])
plt.show()

```

WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.
 WARNING:matplotlib.font_manager:findfont: Font family 'Malgun Gothic' not found.





1.4.4. Trainer 클래스

```
[63] from optimizer import SGD
```

```
[64] from trainer import Trainer
```

```
[65] import spiral
```

```
[68] import two_layer_net
```

```
[69] # 1. 하이퍼파라미터 설정
```

```
max_epoch = 300  
batch_size = 30  
hidden_size = 10  
learning_rate = 1.0
```

```
# 2. 데이터 읽기, 모델과 옵티마이저 생성
```

```
x, t = spiral.load_data()  
model = TwoLayerNet(input_size=2,  
                    hidden_size=hidden_size,  
                    output_size=3)  
optimizer = SGD(lr=learning_rate)
```

```
# 학습에 사용하는 변수
```

```
data_size = len(x)  
max_iters = data_size // batch_size  
total_loss = 0  
loss_count = 0  
loss_list = []
```

```
[70] for epoch in tqdm(range(max_epoch)):
```

```
    # 3. 데이터 셔플링
```

```
    idx = np.random.permutation(data_size)  
    x = x[idx]  
    t = t[idx]
```

```
    for iters in range(max_iters):  
        batch_x = x[iters*batch_size:(iters+1)*batch_size]  
        batch_t = t[iters*batch_size:(iters+1)*batch_size]
```

```
    # 기울기를 구해 매개변수 갱신
```

```
    loss = model.forward(batch_x, batch_t)  
    model.backward()  
    optimizer.update(model.params, model.grads)
```

```
    total_loss += loss  
    loss_count += 1
```

```
# 정기적으로 학습 경과 출력
```

```
if (iters+1) % 10 == 0:  
    avg_loss = total_loss / loss_count  
    print(f'| 에폭 {epoch+1} | 반복{iters+1}/{max_iters} | 손실 {avg_loss:.2f}')  
    loss_list.append(avg_loss)  
    total_loss, loss_count = 0, 0
```



```
[63] from optimizer import SGD
```

```
[64] from trainer import Trainer
```

```
[65] import spiral
```

```
[68] import two_layer_net
```

```
[69] # 1. 하이퍼파라미터 설정
```

```
max_epoch = 300
```

```
batch_size = 30
```

```
hidden_size = 10
```

```
learning_rate = 1.0
```

```
# 2. 데이터 읽기, 모델과 옵티마이저 생성
```

```
x, t = spiral.load_data()
```

```
model = TwoLayerNet(input_size=2,  
                    hidden_size=hidden_size,  
                    output_size=3)
```

```
optimizer = SGD(lr=learning_rate)
```

```
# 학습에 사용하는 변수
```

```
data_size = len(x)
```

```
max_iters = data_size // batch_size
```

```
total_loss = 0
```

```
loss_count = 0
```

```
loss_list = []
```

```
[70] for epoch in tqdm(range(max_epoch)):
```

```
    # 3. 데이터 셔플링
```

```
    idx = np.random.permutation(data_size)
```

```
    x = x[idx]
```

```
    t = t[idx]
```

```
    for iters in range(max_iters):
```

```
        batch_x = x[iters*batch_size:(iters+1)*batch_size]
```

```
        batch_t = t[iters*batch_size:(iters+1)*batch_size]
```

```
        # 기울기를 구해 매개변수 갱신
```

```
        loss = model.forward(batch_x, batch_t)
```

```
        model.backward()
```

```
        optimizer.update(model.params, model.grads)
```

```
        total_loss += loss
```

```
        loss_count += 1
```

```
        # 정기적으로 학습 경과 출력
```

```
        if (iters+1) % 10 == 0:
```

```
            avg_loss = total_loss / loss_count
```

```
            print(f'| 에폭 {epoch+1} | 반복{iters+1}/{max_iters} | 손실 {avg_loss:.2f}')
```

```
            loss_list.append(avg_loss)
```

```
            total_loss, loss_count = 0, 0
```

1.5. 계산 고속화

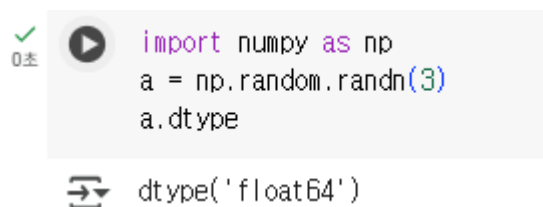
신경망에서는 얼마나 빠르게 계산하는지가 매우 중요

So, 신경망 고속화에 도움이 되는 “비트 정밀도”와 “GPU”에 관해 가볍게 알아보자

1.5.1. 비트 정밀도

넘파이의 부동소수점 수는 기본적으로 64bit 데이터 타입을 사용한다.

사용하는지 확인하는 방법으로 알아보자



넘파이는 64비트 부동소수점 수를 표준으로 사용

BUT, 32비트를 사용하는 이유?

하지만 신경망 추론과 학습은 32비트 부동소수점 수로도 문제없이 수행 가능

메모리 관점에서 보면... 절반의 크기를 절약하는 것이니 항상 32비트가 좋다고 할 수 있다

신경망 계산시 Data를 전송하는 “버스 대역폭”이 병목되는 경우가 有

이때도 데이터 타입이 작은 것이 유리

계산 속도 측면에서도 32 비트 부동소수점 수가 일반적으로 빠르다

⇒ 32bits 로 세팅

```
✓ [59] b = np.random.randn(3).astype(np.float32)  
0초
```

```
✓ [60] b.dtype  
0초
```

```
⇒ dtype('float32')
```

```
✓ [61] c = np.random.randn(3).astype('f')  
0초
```

```
✓ [62] c.dtype  
0초
```

```
⇒ dtype('float32')
```

사실 신경망 추론으로만 한정한다면 16비트를 사용해도 인식률이 괜찮다

BUT, 일반적으로 CPU와 GPU는 연산 자체를 32bits로 수행한다.

⇒ 16비트로 변환해도 계산 자체가 32비트로 이루어져서 처리 속도 측면에서 혜택이 없을 수 있다

⇒ BUT, 학습된 가중치를 저장할 땐 여전히 16비트가 유효하다!

⇒ SO, 학습된 가중치를 저장하는 경우에 한해 16비트 부동소수점 수로 변환하겠다.

1.5.2. GPU(쿠파이)

대량의 곱하기 연산 대부분은 병렬로 계산된다.

이때 CPU보타 GPU가 유리함

```
import cupy as cp
```

```
x = cp.arange(6).reshape(2,3).astype('f')  
x
```

```
array([[0., 1., 2.],  
       [3., 4., 5.]], dtype=float32)
```

그냥 numpy를 cupy로 대체해주기만 하면 끝

1.6. 정리

1. 신경망은 입력층, 은닉층(중간층), 출력층을 지닌다.
2. 완전연결계층에 의해 선형 변환이 이루어지며
3. 활성화 함수에 의해 비선형 변환이 이루어 진다
4. 완전 연결계층이나 미니배치 처리는 행렬로 모아 한꺼번에 계산할 수 있다.
5. 오차역전파법을 사용하여 신경망의 손실에 관한 기울기를 효율적으로 구할 수 있다.
6. 신경망이 수행하는 처리는 계산 그래프로 시각화할 수 있으며, 순전파와 역전파를 이해하는 데 도움 된다.
7. 신경망의 구성요소들을 "계층"으로 모듈화 해두면, 이를 조립하여 신경망을 쉽게 구성할 수 있다.
8. 신경망 고속화에는 GPU를 이용한 병렬 계산과 데이터의 비트 정밀도가 중요하다.